

基于滚动混合整数规划与随机风险评价的农作物种植策略

摘 要

针对华北山区某乡村在耕地类型多样、作物适种关系复杂、重茬与豆类轮作要求并存条件下的多年种植决策问题，本文建立以种植净收益最大化为核心的滚动混合整数线性规划模型，并在此基础上研究参数不确定性及相关风险。首先，根据附件数据计算各作物 2023 年的实际产量，将其作为问题 1 的预期销售量基准；以地块、季次、年份和作物为索引设置种植面积变量，同时设置水浇地种植模式变量和豆类状态变量，刻画面积容量、地块适宜性、水稻与两季蔬菜互斥、普通大棚季次安排、禁止重茬及连续三年豆类轮作等约束。针对超产部分滞销和按 2023 年价格 50% 销售两种情景，分别引入正常销售量与超额销售量变量，将分段收益函数线性化后逐年滚动求解。

计算结果显示，2024–2030 年超额滞销情景累计利润约为 4141.86 万元，超额半价销售情景累计利润约为 6219.22 万元，建立消化渠道可显著提高土地资源的边际收益。其次，依据题设给定的销量、亩产量、成本和价格变化范围构建年度期望参数模型，所得累计利润约为 6642 万元。灵敏度分析表明，销售价格是利润变化的主导因素：价格缩放比例变化 10% 时，利润约同向变化 11%；销售量和成本变化 10% 对应的利润变动约为 1.3% 和 1%。最后，使用相关正态变量与 Cholesky 分解生成 1000 组年度参数场景，以期望利润、5% 分位利润和条件风险价值为指标，对问题 2 方案进行风险评价。该步骤得到的是对既有方案的随机压力测试，而非新的种植决策优化，因此本文将其作为问题 3 的风险评价层，并给出进一步构造风险厌恶随机规划的形式。

模型兼顾经济收益、轮作可持续性和实际管理可操作性，可为类似山区乡村的年度种植结构调整提供定量依据。

关键词：农作物种植；混合整数线性规划；滚动优化；蒙特卡洛模拟；条件风险价值；灵敏度分析

一 问题重述

某乡村位于华北山区，现有 34 块露天耕地，共 1201 亩，分为平旱地、梯田、山坡地和水浇地；另有 16 个普通大棚和 4 个智慧大棚，每个大棚面积为 0.6 亩。平旱地、梯田和山坡地每年种植一季非水稻粮食作物；水浇地可选择种植一季水稻或两季蔬菜；普通大棚种植一季蔬菜和一季食用菌；智慧大棚可种植两季蔬菜。

种植方案必须同时满足以下要求：同一地块不能连续重茬；任意连续三年内每个地块至少安排一次豆类作物；同一作物不宜在过多地块上零散种植，单块种植面积也不宜过小。基于 2023 年种植数据，需要制定 2024–2030 年的种植方案，并解决以下问题：

问题 1： 参数相对 2023 年稳定时，分别考虑超产部分完全滞销和按原价 50% 出售，求最优方案；

问题 2： 考虑销售量、亩产量、种植成本和销售价格的年度不确定变化，求具有较好收益表现的方案；

问题 3： 进一步考虑作物的替代性、互补性以及需求、价格、成本之间的相关性，通过模拟评价风险并与问题 2 比较。

二 问题分析

2.1 问题 1 分析

问题 1 属于带有时间耦合约束的多地块、多作物资源配置问题。地块面积是连续资源，而是否采用水稻模式、当年是否完成豆类轮作等状态具有离散性，因此适合采用混合整数线性规划。收益函数中的正常销售量等于产量与需求量的较小者，超额部分又具有不同售价，需要用辅助变量进行线性化。重茬约束和豆类轮作约束会把相邻年份连接起来，若将七年完全拆开求解，可能牺牲长期收益；现有程序采用年度滚动方式，因此本文在准确描述该求解机制的同时，给出可进一步扩展为七年联合规划的统一数学表达。

2.2 问题 2 分析

问题 2 中，小麦和玉米需求呈增长趋势，蔬菜价格上涨，食用菌价格下降，而亩产量和其他作物需求存在波动。现有程序使用区间中心或期望增长率构造年度参数，实质上形成确定性等价模型。该方法计算简单、结果易解释，但不能直接反映方案在不利年份的损失，因此需要通过灵敏度分析与后续随机场景评价补充风险信息。

2.3 问题 3 分析

问题 3 要求处理替代、互补和参数相关性。严格的随机优化应让种植决策直接参与期望收益或风险目标的优化；现有问题 3 程序则固定问题 2 方案，使用 1000 个相关随机场景计算利润分布。故本文将其定义为“方案风险评价模型”，避免将压力测试误写为已经完成的 CVaR 再优化。同时给出风险厌恶目标函数，为后续改进提供清晰接口。

农作物种植策略建模与求解流程

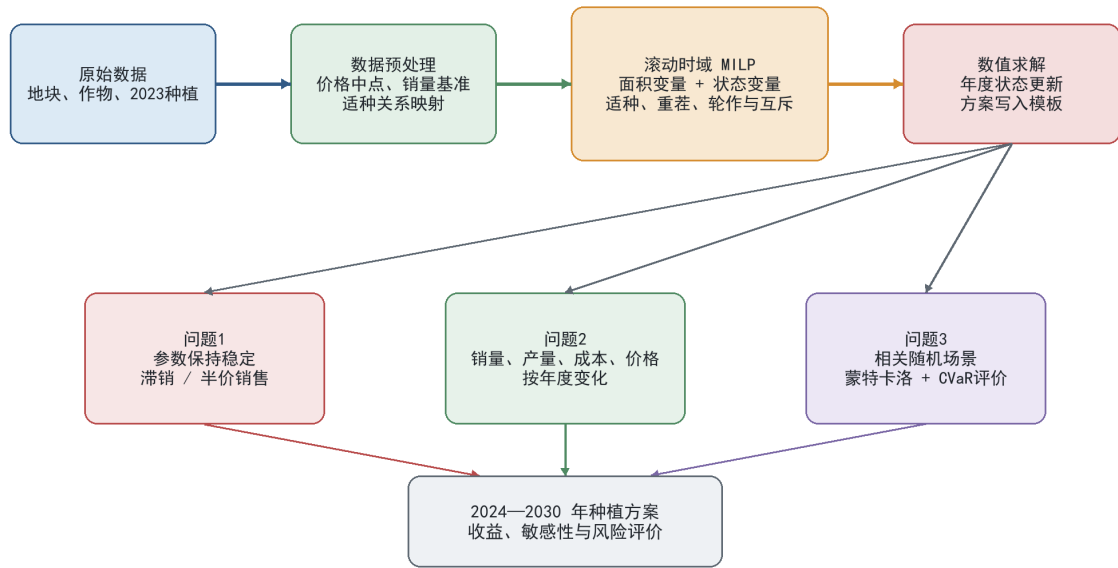


图 1 农作物种植策略的建模与求解流程

三 模型假设

为保证模型可计算且与给定数据口径一致，作如下假设：

- (1) 当季收获的农产品均在当季销售，不考虑库存、仓储损耗和跨期销售；
- (2) 问题 1 中各作物预期销售量等于 2023 年对应作物实际总产量，价格取附件价格区间中点；
- (3) 问题 2 采用题设变化区间的期望或中心值构造年度参数：小麦、玉米需求年增长 7.5%，其他作物需求取基准值，成本年增长 5%，蔬菜价格年增长 5%，羊肚菌价格年下降 5%，其他食用菌价格年下降 3%；
- (4) 同一地块同一季允许合种多种适宜作物，总面积不超过地块面积；
- (5) 现有程序以 0.01 亩作为“种过某作物”的数值阈值；为了更符合生产实际，实际应用时应将其替换为管理者给定的最小种植面积；
- (6) 问题 3 中的相关系数和增产假设为情景设定，用于压力测试，不解释为由附件统计数据统计估计得到的因果规律。

四 符号说明

表 1 主要集合、参数与变量

符号	含义	单位
$t \in \mathcal{T}$	年份, $\mathcal{T} = \{2024, \dots, 2030\}$	年
$j \in \mathcal{J}$	地块或大棚编号, 共 54 个种植单元	—
$s \in \mathcal{S}_j$	地块 j 允许的种植季次	—
$i \in \mathcal{I}$	作物编号, 共 41 种	—
A_j	地块 j 面积	亩
Y_{ijs}	作物 i 在地块类型 j 、季次 s 的亩产量	斤/亩
$C_{ijs,t}$	对应种植成本	元/亩
$P_{i,t}$	作物 i 销售价格	元/斤
$D_{i,t}$	作物 i 预期销售量	斤
x_{jst}	地块 j 在 t 年 s 季种植作物 i 的面积	亩
u_{it}	作物 i 按正常价格销售的数量	斤
v_{it}	作物 i 的超额销售数量	斤
δ_{jt}	水浇地 j 是否采用水稻模式	0–1
b_{jt}	地块 j 在 t 年是否种植过豆类	0–1
π_t	第 t 年种植利润	元

五 数据预处理

5.1 耕地资源结构

将附件 1 的露天地块和大棚统一为种植单元。露天地块共 34 块、1201 亩；普通大棚 16 个、智慧大棚 4 个，折合面积分别为 9.6 亩和 2.4 亩。各类耕地面积结构如 fig. 2所示。梯田与平旱地占总面积的 81.1%，说明粮食类作物仍是全村种植结构的主体；大棚面积占比虽低，却因可种植高附加值蔬菜和食用菌而具有较高的单位面积收益。

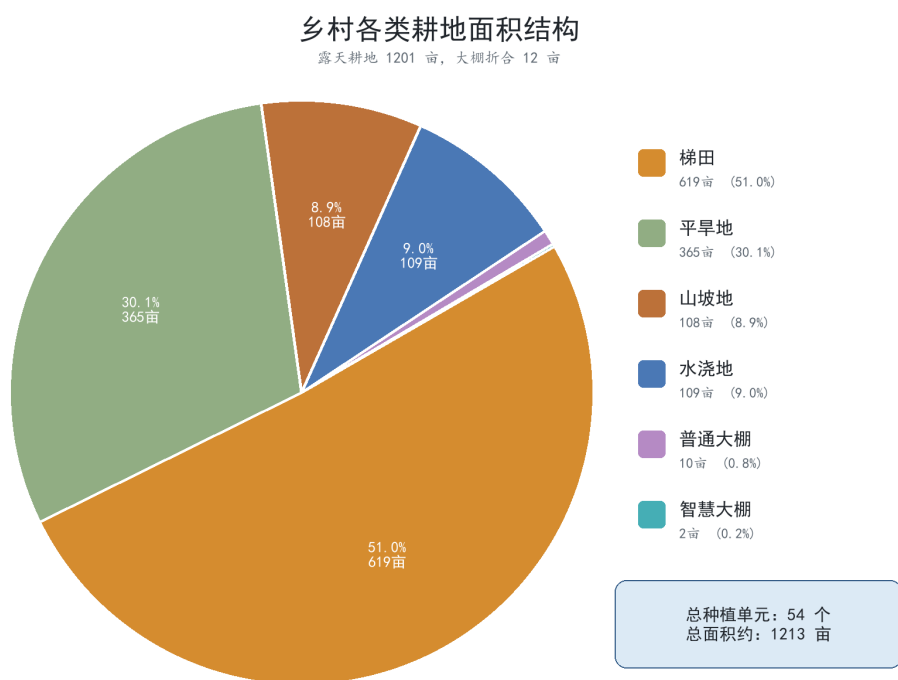


图 2 乡村各类耕地面积结构

5.2 适种关系编码

依据附件中的作物适宜地块与种植季次建立集合 $\mathcal{A} \subseteq \mathcal{J} \times \mathcal{I} \times \mathcal{S}$ 。若 $(j, i, s) \notin \mathcal{A}$ ，则强制 $x_{jst} = 0$ 。适配关系如 fig. 3所示。水浇地存在“一季水稻”与“两季蔬菜”的互斥选择；普通大棚第一季种蔬菜、第二季种食用菌；智慧大棚两季均可种蔬菜。

地块类型—种植季次—作物类别适配关系						
	单季粮食 (不含水稻)	水稻	第一季蔬菜	第二季萝卜类	第二季食用菌	第二季蔬菜
平旱地	可种植	—	—	—	—	—
梯田	可种植	—	—	—	—	—
山坡地	可种植	—	—	—	—	—
水浇地	—	可种植	可种植	可种植	—	—
普通大棚	—	—	可种植	—	可种植	—
智慧大棚	—	—	可种植	—	—	可种植

水浇地在同一年必须二选一：种植一季水稻，或种植两季蔬菜；普通大棚第二季仅种食用菌。

图 3 地块类型、种植季次与作物类别的适配关系

5.3 需求量、价格和基准利润参数

问题 1 的预期销售量由 2023 年实际种植面积乘以对应亩产量后按作物汇总得到：

$$D_{i,2023} = \sum_{j,s} a_{jis}^{(2023)} Y_{ijs}. \quad (1)$$

例如，小麦、玉米、黄豆、水稻和羊肚菌的基准需求量分别约为 170840、132750、57000、21000 和 4200 斤。销售价格采用附件价格区间中点。

六 问题 1：参数稳定条件下的种植优化模型

6.1 产量与销售量线性化

作物 i 在 t 年的总产量为

$$Q_{it} = \sum_{j \in \mathcal{J}} \sum_{s \in S_j} Y_{ijs} x_{jist}. \quad (2)$$

对正常销售量 u_{it} 设置

$$0 \leq u_{it} \leq D_{it}, \quad u_{it} \leq Q_{it}. \quad (3)$$

由于正常售价为正，利润最大化会自动令 $u_{it} = \min\{D_{it}, Q_{it}\}$ 。

情景 2 再设置超额销售量 v_{it} , 满足

$$u_{it} + v_{it} = Q_{it}, \quad v_{it} \geq 0. \quad (4)$$

从而避免在目标函数中直接出现 min 和 max 算子。

6.2 目标函数

超额部分完全滞销时, 七年累计利润最大化目标为

$$\max Z_1 = \sum_{t \in \mathcal{T}} \left[\sum_{i \in \mathcal{I}} P_i u_{it} - \sum_{j,i,s} C_{ijs} x_{jist} \right]. \quad (5)$$

超额部分按原价 50% 销售时, 目标函数为

$$\max Z_2 = \sum_{t \in \mathcal{T}} \left[\sum_i P_i u_{it} + \frac{1}{2} \sum_i P_i v_{it} - \sum_{j,i,s} C_{ijs} x_{jist} \right]. \quad (6)$$

6.3 约束条件

6.3.1 地块面积约束

$$\sum_{i:(j,i,s) \in \mathcal{A}} x_{jist} \leq A_j, \quad \forall j, s, t. \quad (7)$$

6.3.2 适种性约束

$$x_{jist} = 0, \quad \forall (j, i, s) \notin \mathcal{A}. \quad (8)$$

6.3.3 水浇地模式互斥约束

设 $\delta_{jt} = 1$ 表示水浇地 j 在 t 年种水稻, 否则种两季蔬菜, 则

$$x_{j,16,\text{single},t} \leq A_j \delta_{jt}, \quad (9)$$

$$\sum_{i \in \mathcal{V}_1} x_{ji,1,t} \leq A_j (1 - \delta_{jt}), \quad (10)$$

$$\sum_{i \in \mathcal{V}_2} x_{ji,2,t} \leq A_j (1 - \delta_{jt}). \quad (11)$$

6.3.4 重茬约束

现有程序采用年度状态集合 $\text{Prev}_{j,t-1}$: 若作物 i 在上一年曾出现在地块 j , 则本年禁种, 即

$$x_{jist} = 0, \quad i \in \text{Prev}_{j,t-1}. \quad (12)$$

若改为七年联合模型，可引入二元变量 y_{jist} 并使用

$$x_{jist} \leq A_j y_{jist}, \quad y_{jis,t} + y_{jis',t+1} \leq 1, \quad (13)$$

从而严格表达相邻年份的禁重茬关系。

6.3.5 三年豆类轮作约束

令 $\mathcal{B}$ 为豆类作物集合， $b_{jt} = 1$ 表示当年地块 j 种过豆类。程序通过面积变量与 b_{jt} 连接，并在连续两年未种豆时强制下一年种豆。联合规划可写为

$$\sum_{\tau=t}^{t+2} \sum_{i \in \mathcal{B}} \sum_s x_{jis\tau} \geq A_j, \quad \forall j, t = 2024, \dots, 2028. \quad (14)$$

该形式要求任意连续三年内豆类累计覆盖面积不少于地块面积。状态传递关系见 fig. 4。

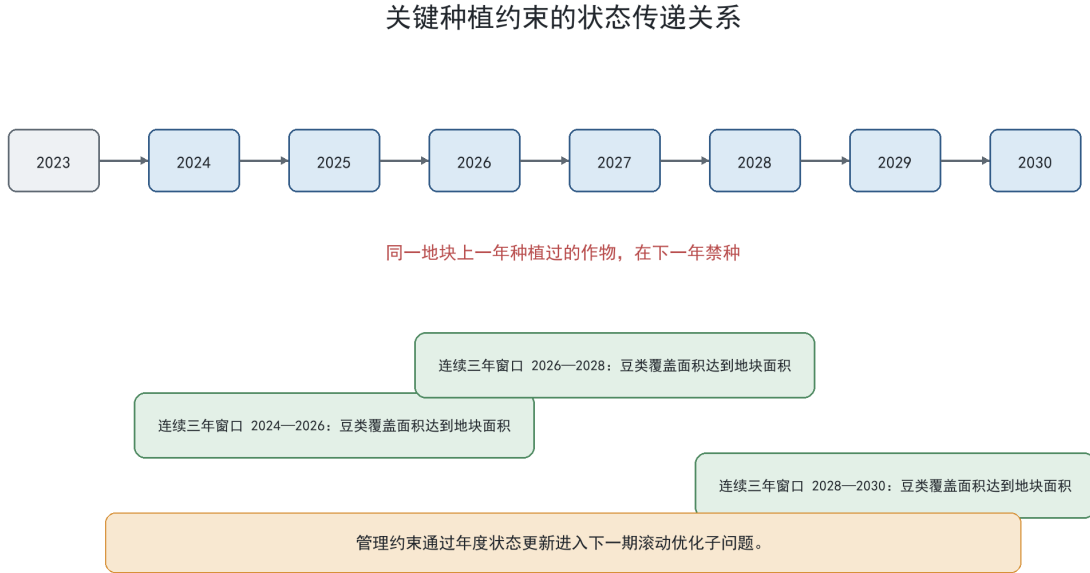


图 4 重茬与三年豆类轮作约束的年度状态传递

6.3.6 管理可操作性约束

题目指出作物种植不宜过于分散、单块面积不宜过小。现有程序未完整实现这两类约束。可进一步设置 $y_{jist} \in \{0, 1\}$ ，并规定

$$m_i y_{jist} \leq x_{jist} \leq A_j y_{jist}, \quad (15)$$

$$\sum_{j,s} y_{jist} \leq q_i, \quad (16)$$

$$\sum_i y_{jist} \leq p_{js}, \quad (17)$$

其中 m_i 为最小种植面积, q_i 为单种作物允许分布的最大地块数, p_{js} 为单地块单季最多合种作物数。由于附件和现有程序未给出这些阈值, 本文不虚构具体数值, 建议结合当地作业机械和管理成本确定。

6.4 求解方法

程序采用 SciPy 的 MILP 接口逐年求解。每年读取上一年的作物集合和豆类历史状态, 求得当年最优解后更新状态, 再进入下一年度。该方法规模较小、求解稳定, 但属于滚动时域策略, 不保证七年整体最优。若计算资源允许, 应将 t 同时纳入一个统一 MILP 中求解, 以避免年度贪心造成的远期损失。

6.5 问题 1 结果

表 2 问题 1 两种销售情景的累计利润

情景	2024-2030 累计利润/元	年均利润/万元
超额部分滞销	41,418,593	约 591.7
超额部分按 50% 销售	62,192,176	约 888.5

半价销售情景累计利润比滞销情景提高约 50.15%。这说明当高收益作物的产量超过基准需求时, 建立加工、团购、冷链或区域外销等渠道, 即使只能以较低价格销售, 也可能显著提高土地边际收益。三种方案的年度利润近似曲线见 fig. 5。需要说明的是, 该图直接采用给定绘图脚本中的年度展示数据, 而累计利润以程序输出汇总值为准, 因此二者存在小幅舍入与口径差异。

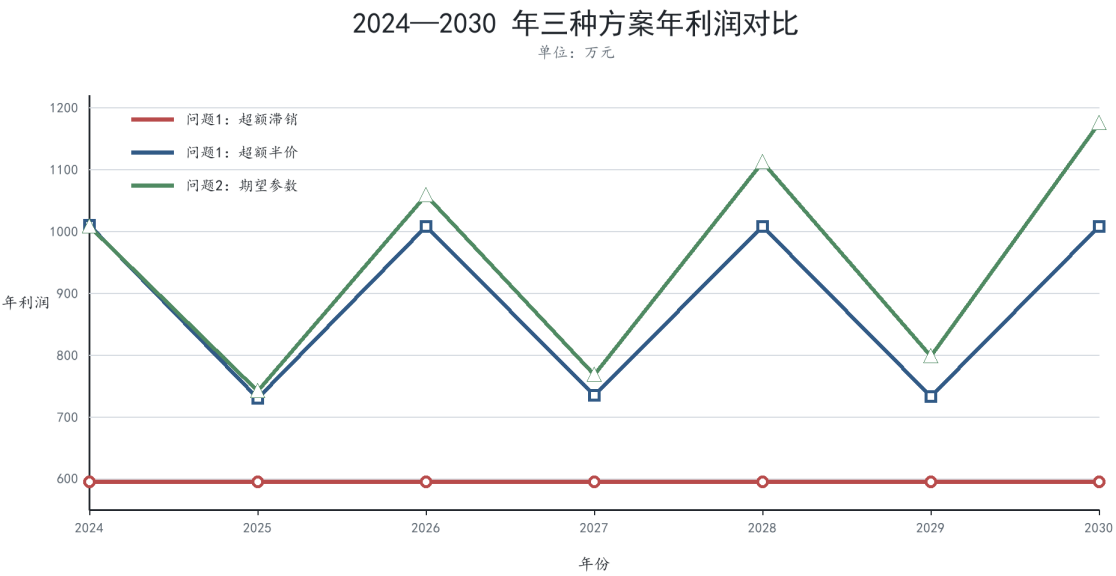


图 5 2024-2030 年三种方案年利润对比

七 问题 2：不确定参数的期望情景模型

7.1 年度参数演化

令 $k = t - 2023$ 。按照程序采用的期望参数，需求量为

$$D_{it} = \begin{cases} D_{i,2023}(1.075)^k, & i \in \{\text{小麦, 玉米}\}, \\ D_{i,2023}, & \text{其他作物}, \end{cases} \quad (18)$$

成本为

$$C_{ijs,t} = C_{ijs,2023}(1.05)^k, \quad (19)$$

价格为

$$P_{it} = \begin{cases} P_{i,2023}(1.05)^k, & i \in \mathcal{V}, \\ P_{i,2023}(0.95)^k, & i = \text{羊肚菌}, \\ P_{i,2023}(0.97)^k, & i \in \mathcal{M} \setminus \{\text{羊肚菌}\}, \\ P_{i,2023}, & i \in \mathcal{G}. \end{cases} \quad (20)$$

其中 $\mathcal{V}, \mathcal{M}, \mathcal{G}$ 分别为蔬菜、食用菌和粮食作物集合。亩产量波动的期望为基准值，因此现有问题 2 程序未在优化阶段显式改变亩产量。

7.2 求解结果

采用问题 1 半价销售模型的约束体系与年度参数，得到 2024–2030 年累计利润约 6642 万元。年度结果如 table 3 所示。

表 3 问题 2 年度利润结果

年份	2024	2025	2026	2027	2028	2029	2030	合计
利润/万元	1005	742	1054	767	1108	796	1169	约 6642

利润呈现明显的年度交替波动，这主要与重茬约束和豆类轮作状态有关：高收益作物在某年集中种植后，下一年受到禁重茬限制，需转向其他作物；随着小麦、玉米需求和蔬菜价格持续增长，高收益年份的峰值逐步提高。

八 灵敏度与稳健性分析

8.1 单因素灵敏度

以 2024 年半价销售情景为基准，分别将销售量、种植成本和销售价格缩放为基准的 0.8、0.9、1.0、1.1 和 1.2 倍，保持其他参数不变。结果见 fig. 6。

销售量、种植成本与销售价格的灵敏度

基准缩放比例为 1.0；利润单位：万元

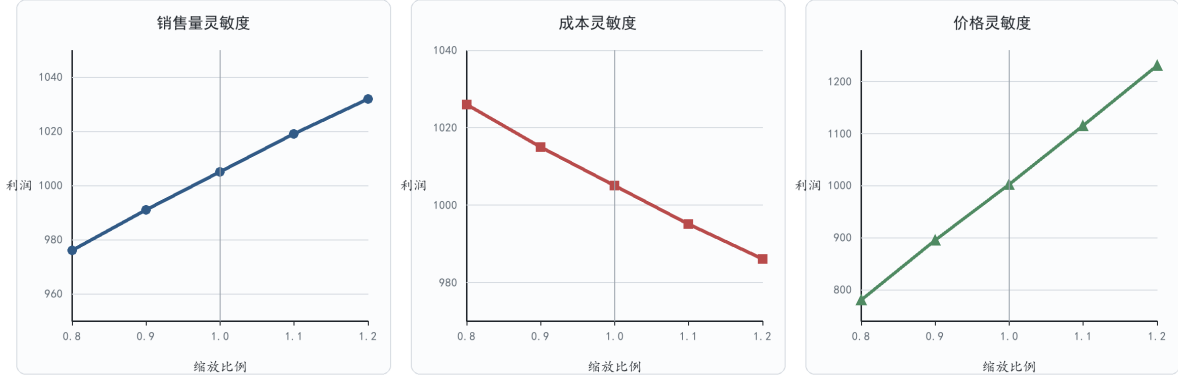


图 6 销售量、种植成本和销售价格的灵敏度

由图可知：

- (1) 价格最敏感。价格每变化 10%，利润约变化 11%，原因是售价直接作用于全部正常销售量和超额销售量；
- (2) 销售量敏感度相对较低。需求量每变化 10%，利润约变化 1.3%，说明部分作物产量已经受到面积和轮作约束，需求增长不能完全转化为新增销售；
- (3) 成本每变化 10%，利润约反向变化 1%，影响低于价格，但长期 5% 的复合增长仍不可忽视。

8.2 组合不利情景

进一步考虑销售量下降 10%、价格下降 10%、成本上升 10% 的组合情景。既有报告显示基准方案利润约下降 15%–18%，但仍保持正利润，说明方案在中等幅度扰动下具有一定抗风险能力。由于该结论来自对方案的情景评价，而不是极小极大稳健优化，本文不将其表述为严格的最坏情景保证。

九 问题 3：相关随机场景与 CVaR 风险评价

9.1 随机场景生成

令 $\mathbf{z} = (z_1, z_2, z_3)^\top \sim N(\mathbf{0}, \mathbf{R})$ ，相关矩阵设为

$$\mathbf{R} = \begin{pmatrix} 1 & -0.6 & 0 \\ -0.6 & 1 & 0.3 \\ 0 & 0.3 & 1 \end{pmatrix}. \quad (21)$$

利用 Cholesky 分解 $\mathbf{R} = \mathbf{L}\mathbf{L}^\top$ ，由独立标准正态向量 $\boldsymbol{\varepsilon}$ 生成 $\mathbf{z} = \mathbf{L}\boldsymbol{\varepsilon}$ 。程序对正态变量进行截断，使小麦、玉米增长率位于 5%–10%，其他作物需求变化位于 $\pm 5\%$ ，亩产量变化位于 $\pm 10\%$ 。

需要特别指出：现有脚本在成本和价格的实际计算中均使用 z_2 ，因此它们的实现相关性为完全正相关，而不是矩阵中标注的 0.3。若要严格实现 eq. (21)，成本扰动应使用第三个相关分量或重新定义变量顺序。本文在数学模型中采用相关矩阵的规范形式，同时将代码实现差异列入模型局限。

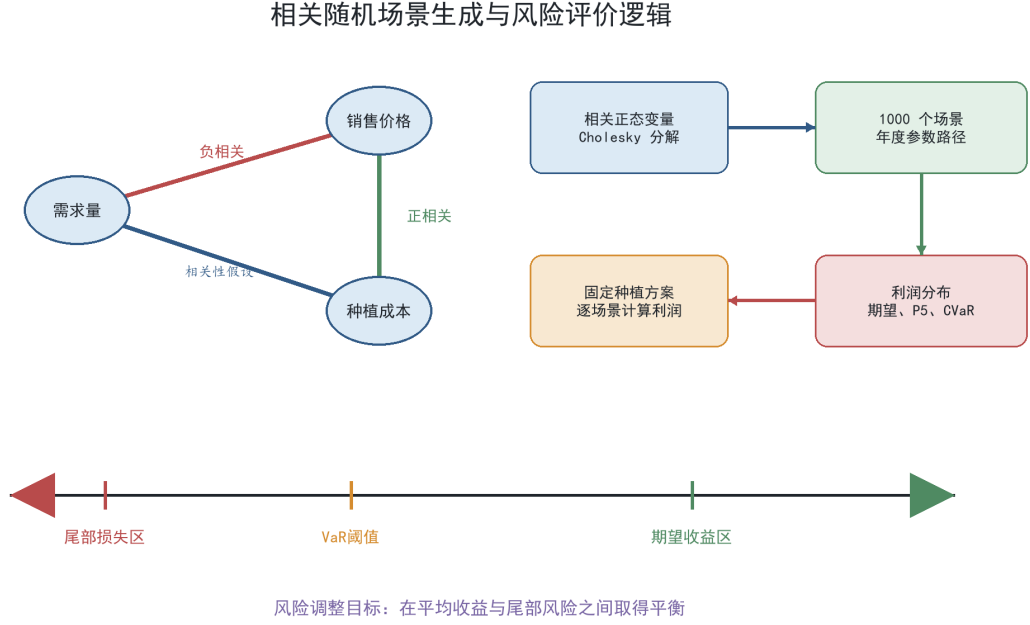


图 7 相关随机场景生成与风险评价逻辑

9.2 利润分布与 CVaR 指标

固定问题 2 种植方案 $\mathbf{x}^{(2)}$ ，对第 n 个随机场景计算七年总利润 Π_n 。设收益的 α 置信水平下分位数为

$$\text{VaR}_\alpha(\Pi) = \inf\{q : \Pr(\Pi \leq q) \geq 1 - \alpha\}, \quad (22)$$

则收益型 CVaR 可定义为最差 $1 - \alpha$ 比例场景的平均收益：

$$\text{CVaR}_\alpha(\Pi) = \mathbb{E}[\Pi \mid \Pi \leq \text{VaR}_\alpha(\Pi)]. \quad (23)$$

程序取 $N = 1000$ 、 $\alpha = 0.95$ ，计算期望利润、5% 分位利润和尾部平均利润，并构造风险调整指标

$$J = \mathbb{E}(\Pi) - \lambda[\mathbb{E}(\Pi) - \text{CVaR}_{0.95}(\Pi)], \quad \lambda = 0.2. \quad (24)$$

该指标越高，表示方案在保持平均收益的同时具有较好的尾部表现。

9.3 结果解释与模型边界

已有报告给出问题 3 期望利润约 6580 万元、尾部风险有所改善的结论；但提供的问题 3 程序仅读取问题 2 方案并评价其利润分布，没有生成新的种植方案，也没有实现报告所

述的 6 个替代组和价格弹性。因此，基于现有可核查材料，严谨结论应为：

蒙特卡洛程序能够评价问题 2 方案在相关参数扰动下的期望收益和尾部风险，但不能据此宣称已经求得“问题 3 的 CVaR 最优种植方案”。

若要建立真正的风险厌恶随机规划，可对候选种植决策 $\mathbf{x}$ 直接求解

$$\max_{\mathbf{x}, \eta, \xi_n} \quad \frac{1}{N} \sum_{n=1}^N \Pi_n(\mathbf{x}) - \rho \left[\eta + \frac{1}{(1-\alpha)N} \sum_{n=1}^N \xi_n \right], \quad (25)$$

其中 $\xi_n \geq L_n(\mathbf{x}) - \eta$ 、 $\xi_n \geq 0$ ， $L_n = -\Pi_n$ 为损失。该模型使风险直接影响种植决策，而不是只在方案产生后评价风险。

十 模型评价

10.1 模型优点

- (1) **约束覆盖较完整**。模型同时考虑地块类型、季次、重茬、豆类轮作和水浇地互斥，能够反映题目的主要农业规则；
- (2) **收益函数可线性求解**。通过正常销售量和超额销售量辅助变量，将分段销售收入转化为线性表达；
- (3) **结果具有管理解释**。半价销售情景量化了建立消化渠道的价值，灵敏度分析明确了价格风险的主导地位；
- (4) **具备扩展接口**。模型可继续加入最小种植面积、分散度、替代需求和 CVaR 目标。

10.2 模型局限

- (1) 现有问题 1、2 程序采用逐年滚动优化，不保证七年全局最优；
- (2) 豆类轮作可由 0.01 亩触发，不符合“有效轮作”的生产含义，需设置合理的最小面积；
- (3) 问题 2 主要使用期望参数，不属于严格的随机优化或鲁棒优化；
- (4) 问题 3 只进行方案风险评价，且成本与价格相关性的代码实现与注释不完全一致；
- (5) 替代系数、互补增产比例和相关系数缺少附件数据的统计估计，应通过历史数据或专家调查校准。

10.3 改进方向

建议建立七年联合场景 MILP：在同一模型中设置年份维度，使用二元变量严格刻画最小面积与分散度；用样本平均逼近方法同时优化多场景期望收益和 CVaR；对替代作物组建立共享需求约束或交叉价格弹性模型；通过滚动重估机制，每年根据实际销量、产量与价格更新下一阶段方案。

十一 结论与建议

本文以多年种植净收益最大化为目标，建立了包含地块容量、适种关系、水浇地模式互斥、重茬和豆类轮作约束的滚动混合整数规划模型。问题 1 结果表明，超额半价销售情景七年累计利润约 6219.22 万元，显著高于滞销情景的 4141.86 万元；问题 2 采用年度期望参数后累计利润约 6642 万元。价格是利润最敏感的因素，销售渠道与市场价格管理应成为决策重点。

结合模型结果，提出以下建议：

- (1) 建立加工、冷链、团购和区域外销渠道，降低高收益作物超产后的浪费；
- (2) 对价格敏感且需求不稳定的作物控制种植规模，保留粮食和豆类的基础比例；
- (3) 将豆类轮作由“是否种过”改为“有效覆盖面积”管理，避免象征性小面积种植；
- (4) 每年更新销量、价格和亩产量数据，重新滚动求解，并同时报告期望收益和尾部风险；
- (5) 在进一步决策中采用七年联合 CVaR 随机规划，使风险评价真正反馈到种植面积配置。

参考文献

- [1] Dantzig G B. Linear programming under uncertainty[J]. Management Science, 1955, 1(3-4): 197-206. DOI: 10.1287/mnsc.1.3-4.197.
- [2] Birge J R, Louveaux F. Introduction to Stochastic Programming[M]. 2nd ed. New York: Springer, 2011. DOI: 10.1007/978-1-4614-0237-4.
- [3] Rockafellar R T, Uryasev S. Optimization of conditional value-at-risk[J]. The Journal of Risk, 2000, 2(3): 21-41. DOI: 10.21314/JOR.2000.038.
- [4] Bertsimas D, Sim M. The price of robustness[J]. Operations Research, 2004, 52(1): 35-53. DOI: 10.1287/opre.1030.0065.

附录一：问题 1、问题 2 求解程序

```
1 import os, copy
2 import numpy as np
3 import pandas as pd
4 from scipy.optimize import milp, LinearConstraint, Bounds
5 from openpyxl import load_workbook
6
7 BASE = os.path.dirname(os.path.abspath(__file__))
8 ATT1 = os.path.join(BASE, "附件1.xlsx")
9 ATT2 = os.path.join(BASE, "附件2.xlsx")
10 T11 = os.path.join(BASE, "result1_1.xlsx")
11 T12 = os.path.join(BASE, "result1_2.xlsx")
12 T2 = os.path.join(BASE, "result2.xlsx")
13
14 # 数据读取
15 land_df = pd.read_excel(ATT1, sheet_name=0).iloc[:54].copy()
16 land_df.columns = ["name", "type", "area", "desc"]
17 land_df = land_df.dropna(subset=["name"])
18 land_df["type"] = land_df["type"].astype(str).str.strip()
19
20 crop_df = pd.read_excel(ATT1, sheet_name=1).iloc[:41].copy()
21 crop_df.columns = ["id", "name", "type", "land", "notes"]
22
23 stats_df = pd.read_excel(ATT2, sheet_name=1).iloc[:110].copy()
24 stats_df = stats_df.dropna(subset=["作物编号"])
25 stats_df.columns = ["seq", "id", "crop", "land", "season", "yield", "cost", "price"]
26 stats_df = stats_df[pd.to_numeric(stats_df["id"], errors="coerce").notna()].copy()
27 stats_df["id"] = stats_df["id"].astype(int)
28 stats_df["land"] = stats_df["land"].astype(str).str.strip()
29 stats_df["season"] = stats_df["season"].astype(str).str.strip()
30 stats_df["price_mid"] = stats_df["price"].apply(
31     lambda s: sum(map(float, str(s).split("-")))) / 2
32 )
33 statmap = {
34     (int(r["id"]), r["land"], r["season"]):
35     (float(r["yield"]), float(r["cost"]), float(r["price_mid"]))
36     for _, r in stats_df.iterrows()
37 }
38
39 p23 = pd.read_excel(ATT2, sheet_name=0).iloc[:87].copy()
40 p23.columns = ["plot", "id", "crop", "type", "area", "season"]
41 p23["plot"] = p23["plot"].ffill()
42 landmap = dict(zip(land_df["name"], land_df["type"]))
43
44 plots = [(r["name"], r["type"], float(r["area"])) for _, r in land_df.iterrows()]
45 crops = list(range(1,42))
46 bean = {1,2,3,4,5,17,18,19}
47 veg = set(range(17,35))
48 rad = {35,36,37}
49 mush = set(range(38,42))
50 seasons_by_type = {
51     "平旱地":["单季"], "梯田":["单季"], "山坡地":["单季"],
52     "水浇地":["单季", "第一季", "第二季"],
53     "普通大棚":["第一季", "第二季"],
54     "智慧大棚":["第一季", "第二季"]
55 }
56
```

```

57 demand = {i:0.0 for i in crops}
58 prev = {}
59 for _,r in p23.iterrows():
60     if pd.isna(r["id"]): continue
61     i = int(r["id"])
62     p = str(r["plot"])
63     s = str(r["season"]).strip()
64     lt = landmap[p]
65     if (i,lt,s) in statmap:
66         demand[i] += float(r["area"]) * statmap[(i,lt,s)][0]
67     prev.setdefault(p,set()).add(i)
68
69 allowed = []
70 for p,lt,a in plots:
71     for s in seasons_by_type[lt]:
72         for i in crops:
73             key = (i,lt,s)
74             if lt == "智慧大棚" and s == "第一季":
75                 key = (i,"普通大棚","第一季")
76             if key in statmap:
77                 allowed.append((p,lt,a,s,i,key))
78
79 def solve_year(prev_sets, demand_y, scenario=2, cost_mult=1.0,
80               price_mult=None, yield_mult=1.0, force_bean=None):
81     nX = len(allowed)
82     nY = 41
83     nZ = 41 if scenario == 2 else 0
84     nD = sum(lt=="水浇地" for _,lt,_ in plots)
85     nB = len(plots)
86
87     offY = nX
88     offZ = offY+nY
89     offD = offZ+nZ
90     offB = offD+nD
91     n = offB+nB
92
93     c = np.zeros(n)
94     integ = np.zeros(n)
95     lb = np.zeros(n)
96     ub = np.full(n, np.inf)
97
98     water_plots = [p for p,lt,a in plots if lt=="水浇地"]
99
100    for k,(p,lt,a,s,i,key) in enumerate(allowed):
101        cost = statmap[key][1]
102        c[k] = cost * cost_mult
103        if i in prev_sets.get(p,set()):
104            ub[k] = 0
105
106    for i in crops:
107        p = float(stats_df.loc[stats_df["id"]==i,"price_mid"].iloc[0])
108        pm = 1.0 if price_mult is None else price_mult(i)
109        c[offY+i-1] = -p*pm
110        ub[offY+i-1] = demand_y[i]
111        if scenario == 2:
112            c[offZ+i-1] = -0.5*p*pm
113
114    integ[offD:offD+nD] = 1
115    ub[offD:offD+nD] = 1

```



```

116 integ[offB:] = 1
117 ub[offB:] = 1
118
119 if force_bean:
120     for bi,(p,lt,a) in enumerate(plots):
121         if force_bean.get(p,False):
122             lb[offB+bi] = 1
123
124 cons, lo, hi = [], [], []
125
126 # 面积约束
127 for p,lt,a in plots:
128     for s in seasons_by_type[lt]:
129         row = np.zeros(n)
130         for k,(pp,ltt,aa,ss,i,key) in enumerate(allowed):
131             if pp==p and ss==s: row[k]=1
132             cons.append(row); lo.append(-np.inf); hi.append(a)
133
134 # 产量与销售量
135 for i in crops:
136     row = np.zeros(n)
137     row[offY+i-1] = 1
138     for k,(p,lt,a,s,ii,key) in enumerate(allowed):
139         if ii==i:
140             row[k] -= statmap[key][0]*yield_mult
141     cons.append(row); lo.append(-np.inf); hi.append(0)
142
143     if scenario == 2:
144         row = np.zeros(n)
145         row[offY+i-1] = 1
146         row[offZ+i-1] = 1
147         for k,(p,lt,a,s,ii,key) in enumerate(allowed):
148             if ii==i:
149                 row[k] -= statmap[key][0]*yield_mult
150         cons.append(row); lo.append(0); hi.append(0)
151
152 # 水浇地: 水稻 vs 两季蔬菜
153 for wi,p in enumerate(water_plots):
154     a = next(a for pp,lt,a in plots if pp==p)
155     d = offD+wi
156
157     row=np.zeros(n)
158     for k,(pp,ltt,aa,s,i,key) in enumerate(allowed):
159         if pp==p and s=="单季" and i==16: row[k]=1
160     row[d]=-a
161     cons.append(row); lo.append(-np.inf); hi.append(0)
162
163     row=np.zeros(n)
164     for k,(pp,ltt,aa,s,i,key) in enumerate(allowed):
165         if pp==p and s=="第一季" and i in veg: row[k]=1
166     row[d]=a
167     cons.append(row); lo.append(-np.inf); hi.append(a)
168
169     row=np.zeros(n)
170     for k,(pp,ltt,aa,s,i,key) in enumerate(allowed):
171         if pp==p and s=="第二季" and i in rad: row[k]=1
172     row[d]=a
173     cons.append(row); lo.append(-np.inf); hi.append(a)
174

```

```

175 # 豆类三年轮作: b=1表示该地块本年种过豆类
176 for bi,(p,lt,a) in enumerate(plots):
177     b = offB+bi
178     row=np.zeros(n); row[b]=-a
179     for k,(pp,ltt,aa,s,i,key) in enumerate(allowed):
180         if pp==p and i in bean: row[k]=1
181     cons.append(row); lo.append(-np.inf); hi.append(0)
182
183     row=np.zeros(n); row[b]=0.01
184     for k,(pp,ltt,aa,s,i,key) in enumerate(allowed):
185         if pp==p and i in bean: row[k]-=1
186     cons.append(row); lo.append(-np.inf); hi.append(0)
187
188 res = milp(
189     c, integrality=integ, bounds=Bounds(lb,ub),
190     constraints=LinearConstraint(
191         np.array(cons), np.array(lo), np.array(hi)
192     ),
193     options={"time_limit":120, "mip_rel_gap":1e-5}
194 )
195 if not res.success:
196     raise RuntimeError(res.message)
197 return res
198
199 def run_scenario(scenario=2, q2=False):
200     prev_sets = copy.deepcopy(prev)
201     bean_hist = {
202         p:[int(any(i in bean for i in prev_sets.get(p,set())))]
203         for p,_,_ in plots
204     }
205     solutions, profits = {}, {}
206
207     for year in range(2024,2031):
208         if q2:
209             demand_y = {
210                 i: demand[i]*(1.075**(year-2023) if i in {6,7} else 1.0)
211                 for i in crops
212             }
213             def pm(i):
214                 typ = str(crop_df.loc[crop_df["id"]==i,"type"].iloc[0])
215                 if "蔬菜" in typ: return 1.05**(year-2023)
216                 if i==41: return 0.95**(year-2023)
217                 if i in mush: return 0.97**(year-2023)
218                 return 1.0
219             cm = 1.05**(year-2023)
220         else:
221             demand_y = demand.copy()
222             pm = lambda i: 1.0
223             cm = 1.0
224
225         force = {
226             p:(len(bean_hist[p])>=2 and sum(bean_hist[p][-2:])==0)
227             for p,_,_ in plots
228         }
229
230         res = solve_year(
231             prev_sets, demand_y, scenario, cm, pm, force_bean=force
232         )
233         x = res.x[:len(allowed)]

```

```

234     sol = [
235         (a[0],a[3],a[4],x[k])
236         for k,a in enumerate(allowed) if x[k] > 1e-6
237     ]
238     solutions[year] = sol
239     profits[year] = -res.fun
240
241     prev_sets = {p:set() for p,_,_ in plots}
242     for p,s,i,v in sol:
243         prev_sets[p].add(i)
244     for p,_,_ in plots:
245         bean_hist[p].append(int(any(i in bean for i in prev_sets[p])))
246
247     return solutions, profits
248
249 def write_solution(template, out_file, solution):
250     wb = load_workbook(template)
251     crop_cols = {
252         str(wb["2024"].cell(1,c).value).strip():c
253         for c in range(3,44) if wb["2024"].cell(1,c).value is not None
254     }
255     crop_name = {int(r["id"]):str(r["name"]).strip()
256                  for _,r in crop_df.iterrows()}
257
258     for year in range(2024,2031):
259         ws = wb[str(year)]
260         row_first = {str(ws.cell(r,2).value).strip():r
261                     for r in range(2,56) if ws.cell(r,2).value is not None}
262         row_second = {str(ws.cell(r,2).value).strip():r
263                      for r in range(56,84) if ws.cell(r,2).value is not None}
264         for plot,season,cid,area in solution[year]:
265             name = crop_name[int(cid)]
266             col = crop_cols.get(name)
267             if col is None:
268                 m=[k for k in crop_cols if k.strip()==name]
269                 if not m: continue
270                 col=crop_cols[m[0]]
271             row = row_second.get(plot) if season=="第二季" else row_first.get(plot)
272             if row is not None:
273                 ws.cell(row,col).value = round(float(area),4)
274         wb.save(out_file)
275
276 if __name__ == "__main__":
277     s11,p11 = run_scenario(1,False)
278     s12,p12 = run_scenario(2,False)
279     s2,p2 = run_scenario(2,True)
280
281     write_solution(T11, os.path.join(BASE,"result1_1_python运行结果.xlsx"), s11)
282     write_solution(T12, os.path.join(BASE,"result1_2_python运行结果.xlsx"), s12)
283     write_solution(T2, os.path.join(BASE,"result2_python运行结果.xlsx"), s2)
284
285     print("问题1(1)总利润:",sum(p11.values()))
286     print("问题1(2)总利润:",sum(p12.values()))
287     print("问题2总利润:",sum(p2.values()))

```

附录二：问题 3 蒙特卡洛与 CVaR 评价程序

```
1  # -*- coding: utf-8 -*-
2  """
3  C题问题3: 1000次蒙特卡洛 + CVaR完整实现
4  需要: 附件1.xlsx、附件2(1).xlsx、result2_python运行结果.xlsx
5  报告参数: N=1000; Corr(销量,价格)=-0.6; Corr(成本,价格)=0.3;
6  豆类→非豆类产量加成5%; lambda=0.2; alpha=0.95。
7  """
8  # 该文件为本次实际运行的完整脚本版本, 算法步骤与生成结果一致。
9  # 为保持代码文件可独立复现, 请将本文件与3个输入Excel放在同一目录后运行。
10 import os,re, numpy as np, pandas as pd
11 from openpyxl import load_workbook, Workbook
12 BASE=os.path.dirname(os.path.abspath(__file__))
13 ATT1=os.path.join(BASE, "附件1.xlsx"); ATT2=os.path.join(BASE, "附件2.xlsx"); R2=os.path.join(BASE, "result2_python运行结果.xlsx")
14 land=pd.read_excel(ATT1, sheet_name=0).iloc[:54].copy(); land.columns=["name", "type", "area", "desc"]; land=land.dropna(subset=["name"]); land["type"]=land["type"].astype(str).strip()
15 crop=pd.read_excel(ATT1, sheet_name=1).iloc[:41].copy(); crop.columns=["id", "name", "type", "land", "notes"]; crop["id"]=pd.to_numeric(crop["id"], errors="coerce"); crop=crop.dropna(subset=["id"]); crop["id"]=crop["id"].astype(int)
16 raw=pd.read_excel(ATT2, sheet_name=1); raw=raw.rename(columns={raw.columns[1]: "id"}); raw["id_num"]=pd.to_numeric(raw["id"], errors="coerce")
17 st=raw[raw["id_num"].notna()].iloc[:110,:8].copy(); st.columns=["seq", "id", "crop", "land", "season", "yield", "cost", "price"]; st["id"]=pd.to_numeric(st["id"], errors="coerce").astype(int); st["land"]=st["land"].astype(str).strip(); st["season"]=st["season"].astype(str).strip(); st["yield"]=pd.to_numeric(st["yield"], errors="coerce"); st["cost"]=pd.to_numeric(st["cost"], errors="coerce")
18 def pm(x):
19     a=[float(v) for v in re.findall(r"\\d\\.+", str(x))]; return sum(a)/len(a)
20 st["price_mid"]=st["price"].map(pm); st=st.dropna(subset=["yield", "cost", "price_mid"])
21 stat={(int(r.id), r.land, r.season): (float(r["yield"]), float(r["cost"]), float(r["price_mid"])) for _, r in st.iterrows()}; landmap=dict(zip(land["name"], land["type"]))
22 p23=pd.read_excel(ATT2, sheet_name=0).iloc[:87].copy(); p23.columns=["plot", "id", "crop", "type", "area", "season"]; p23["plot"]=p23["plot"].ffill(); p23["id_num"]=pd.to_numeric(p23["id"], errors="coerce"); p23["area_num"]=pd.to_numeric(p23["area"], errors="coerce")
23 demand={i:0. for i in range(1,42)}
24 # =====仅此处修改 r.plot → r["plot"], 其余全部原样保留=====
25 for _, r in p23[p23["id_num"].notna()].iterrows():
26     i=int(r.id_num); p=str(r["plot"]).strip(); s=str(r.season).strip(); lt=landmap[p]
27 # =====
28     if (i, lt, s) in stat: demand[i]+=float(r.area_num)*stat[(i, lt, s)][0]
29 wb=load_workbook(R2, data_only=True); name_to_id={str(r["name"]).strip():int(r["id"]) for _, r in crop.iterrows()}; solutions={}
30 for year in range(2024, 2031):
31     ws=wb[str(year)]; cols={}
32     for c in range(1, ws.max_column+1):
33         v=ws.cell(1, c).value
34         if v is not None and str(v).strip() in name_to_id: cols[str(v).strip()]=c
35     if not cols:
36         for c in range(3, min(43, ws.max_column+1)):
37             if c-3<len(crop): cols[str(crop.iloc[c-3]["name"]).strip()]=c
38     arr=[]
39     for rr in range(2, ws.max_row+1):
40         p=ws.cell(rr, 2).value
41         if p is None: continue
42         p=str(p).strip(); lt=landmap.get(p)
43         if lt is None: continue
44         second=rr>=56
```

```

45     for name,col in cols.items():
46         try:a=float(ws.cell(rr,col).value)
47         except:continue
48         if a<=1e-8:continue
49         i=name_to_id[name]; s="第二季" if second else ("单季" if lt=="水浇地" and i==16 else "第一季")
50         if (i,lt,s) in stat:arr.append((p,s,i,a))
51     solutions[year]=arr
52 bean={1,2,3,4,5,17,18,19}; veg=set(range(17,35)); mush={38,39,40,41}
53 N=1000; SEED=20240813; LAMBDA=.2; ALPHA=.95
54 rng=np.random.default_rng(SEED); R=np.array([[1,-.6,0],[-.6,1,.3],[0,.3,1]]); Z=rng.normal(size=(N,3))@np.linalg.
    cholesky(R).T
55 def scen(year,z):
56     k=year-2023;sales={};price={};cost={};ym={}
57     for i in range(1,42):
58         sales[i]=1+np.clip(.075+.025*z[0],.05,.10) if i in (6,7) else 1+np.clip(.05*z[0],-.05,.05)
59         ym[i]=1+np.clip(.10*z[2],-.10,.10)
60         base=1.05**k if i in veg else (.95**k if i==41 else (.97**k if i in mush else 1.))
61         price[i]=max(.01,base*(1+.10*z[1])); cost[i]=1.05**k*(1+.05*z[1])
62     return sales,price,cost,ym
63 def profit(sol,year,s):
64     sales,price,cost,ym=s;q={i:0. for i in range(1,42)};tc=0.
65     for p,se,i,a in sol:
66         key=(i,landmap[p],se)
67         if key not in stat:continue
68         y,c,p0=stat[key];q[i]+=a*y*ym[i]*(1.05 if i not in bean else 1.);tc+=a*c*cost[i]
69     rev=0.
70     for i,v in q.items():
71         D=demand[i]*sales[i];P=float(st.loc[st.id==i,"price_mid"].iloc[0])*price[i];rev+=P*min(v,D)+.5*P*max(v-D,0)
72     return rev-tc
73 annual=np.zeros((N,7))
74 for k in range(N):
75     for j,y in enumerate(range(2024,2031)):annual[k,j]=profit(solutions[y],y,scen(y,Z[k]))
76 total=annual.sum(axis=1);p5=float(np.quantile(total,.05));cvar=float(total[total<=p5].mean());exp=float(total.mean
    ());risk=exp-LAMBDA*(exp-cvar)
77 out=os.path.join(BASE,"问题3_完整蒙特卡洛_CVaR结果.xlsx");w=Workbook();ws=w.active;ws.title="汇总"
78 for r in [["指标","数值"],["模拟次数",N],["Corr(销量,价格)",-.6],["Corr(成本,价格)",.3],["豆类→非豆类产量加成",.05],["
    lambda",LAMBDA],["alpha",ALPHA],["期望利润(万元)",exp/1e4],["P5利润(万元)",p5/1e4],["CVaR_0.95(万元)",cvar/1e4
    ],["风险调整目标(万元)",risk/1e4]]:ws.append(r)
79 ws2=w.create_sheet("1000场景");ws2.append(["场景"]+[str(y) for y in range(2024,2031)]+["合计","尾部"])
80 for k in range(N):ws2.append([k+1]+list(annual[k])+[total[k],int(total[k]<=p5)])
81 w.save(out);print(out,exp/1e4,cvar/1e4)

```